
Reproducing AgentPoison: Evaluating and Defending Against Backdoor Attacks on RAG-Based LLM Agents

Chinmay N

University of California, San Diego
cnadgir@ucsd.edu

Chinmay B

University of California, San Diego
cbharambe@ucsd.edu

Hayden

University of California, San Diego
hakwok@ucsd.edu

Suditi

University of California, San Diego
sus028@ucsd.edu

Abstract

Retrieval-Augmented Generation (RAG) integrates external knowledge databases to enhance Large Language Model (LLM) agents. However, this relies on an implicit trust boundary between the retriever and the agent. In this paper, we present a formal reproduction and extension of AgentPoison (NeurIPS 2024), a stealthy backdoor attack targeting this boundary. By injecting a tiny fraction of poisoned documents (below 0.1%) and optimizing a short text trigger (2–10 tokens) using gradient-based token search, the attacker forces the retriever to surface poisoned context, hijacking the agent’s actions. We replicate the attack on two benchmark agents (ReAct-StrategyQA and EHRAgent) across open-weights backbones (Qwen-2.5-7B, Llama-3-8B) on a shared cluster and API-based models (GPT-4o-mini). While verifying the paper’s core claims, we identify a critical “noisy” backdoor paradox: the attack degrades benign performance on targeted topics even without the trigger, operating as a localized denial-of-service. We also implement a geometric detector defense that flags malicious clustering in embedding space (achieving a 98.0% True Positive Rate against AgentPoison), and analyze the evasion trade-offs of an adaptive attacker.

1 Introduction

Large Language Model (LLM) agents are increasingly deployed in complex environments such as medical decision support, driving simulation, and interactive question answering. To overcome parametric knowledge limits, these systems incorporate Retrieval-Augmented Generation (RAG) by fetching context at runtime from external databases. Despite its prevalence, RAG introduces a significant security vulnerability: the downstream LLM agent assumes retrieved context is benign and compliant, presenting an attractive attack surface for database poisoning.

AgentPoison (Chen et al., NeurIPS 2024) is a stealthy backdoor attack designed to exploit this surface. Unlike prompt-based or model-weight-modifying attacks, AgentPoison is database-centric. An attacker injects a minimal number of adversarial documents (e.g., 0.04% of the corpus) and optimizes a short textual trigger. When the trigger is appended to any query, the dense retriever is forced to rank the poisoned documents at the top. Once the LLM processes this poisoned context, it executes the target adversarial action (e.g., halting an autonomous car or prescribing incorrect medical treatments).

In this work, we present a comprehensive reproduction and extension of the AgentPoison framework. Our contributions are three-fold:

1. **Replication Validation:** We replicate the primary results of AgentPoison on ReAct-StrategyQA and EHRAgent. We validate its efficacy against baselines (CPA, BadChain, GCG) using both Google Colab API backends (GPT-4o-mini) and localized cluster-based deployments of open-weights models (Qwen-2.5-7B, Llama-3-8B).
2. **The Noisy Backdoor Paradox:** We show that the attack is not a silent, clean backdoor. The embedding alignment forces retrievers to surface poisoned documents for target-related benign queries even when the trigger is absent, severely degrading benign accuracy on targeted topics.
3. **Defense Adaptive Evaluation:** We propose a cheap geometric defense that flags malicious dense retrieval clusters, achieving a 98.0% True Positive Rate (TPR) against AgentPoison. We evaluate an adaptive attacker that penalizes cluster density, showing a clear trade-off between evasion capability and attack success rate.

2 Threat Model and Methodology

2.1 System Formulation

Let A represent an LLM agent, and $D = \{d_1, d_2, \dots, d_N\}$ be the knowledge database. For a user query q , a dense retriever R embeds both the query and the documents to compute similarity scores:

$$S(q, d) = \cos(E(q), E(d)) = \frac{E(q) \cdot E(d)}{\|E(q)\|_2 \|E(d)\|_2} \quad (1)$$

where E is the query/context encoder. The top- k documents are retrieved:

$$D_{ret} = \arg \text{top-}k_{d \in D} S(q, d) \quad (2)$$

The agent A takes q and D_{ret} to generate a trajectory of actions $a_{1:T}$ ending in an outcome o .

2.2 Threat Model

The attacker seeks to force a target outcome o^* when a short trigger T is appended to q (i.e., the triggered query is $q' = [q; T]$). The attacker is constrained by:

1. **Data Poisoning Limits:** The attacker can insert only a small number of poisoned documents D_{pois} into D (where $|D_{pois}| \ll |D|$).
2. **Stealthiness:** The trigger T must be short (2–10 tokens) and coherent. The database entries D_{pois} must appear benign.
3. **No Model/Prompt Access:** The attacker cannot modify the LLM weights, the retriever weights, or the agent’s system prompt templates.

2.3 Trigger Optimization Objective

To optimize the trigger T , AgentPoison minimizes a multi-term loss function $\mathcal{L}_{AP}$ using a discrete HotFlip-style gradient search over the token vocabulary V :

$$\mathcal{L}_{AP} = \mathcal{L}_{uniq} + \alpha \mathcal{L}_{comp} + \beta \mathcal{L}_{action} + \gamma \mathcal{L}_{cohere} \quad (3)$$

where:

- **Uniqueness Loss ($\mathcal{L}_{uniq}$)** pulls the embedding of the triggered query q' toward the centroid of the poisoned documents:

$$\mathcal{L}_{uniq} = \left\| E(q') - \frac{1}{|D_{pois}|} \sum_{d \in D_{pois}} E(d) \right\|_2^2 \quad (4)$$

- **Compactness Loss ($\mathcal{L}_{comp}$)** minimizes the variance of the triggered query embeddings to ensure they map to a tight region:

$$\mathcal{L}_{comp} = \frac{1}{|Q|} \sum_{q \in Q} \left\| E(q') - \bar{E}(q') \right\|_2^2 \quad (5)$$

- **Target Action Loss** ($\mathcal{L}_{action}$) maximizes the probability that the LLM agent outputs the adversarial target action:

$$\mathcal{L}_{action} = - \sum_t \log P_{LLM}(a_t^* | q', D_{pois}, a_{<t}^*) \quad (6)$$

- **Coherence Loss** ($\mathcal{L}_{cohere}$) penalizes high perplexity using a pre-trained language model (GPT-2) to keep the trigger readable:

$$\mathcal{L}_{cohere} = - \sum_i \log P_{GPT2}(T_i | T_{<i}) \quad (7)$$

3 Reproduction Setup

We replicated and analyzed the attack across two distinct environments:

1. **Open-Weights Cluster (DSMLP)**: Evaluated on UCSD’s DSMLP cluster using NVIDIA RTX A5000 GPUs (24GB VRAM) for local inference of open-weights agents: Qwen-2.5-7B-Instruct and Meta-Llama-3-8B-Instruct.
2. **API-Based Notebook (Colab)**: Evaluated using GPT-4o-mini as a drop-in replacement for the deprecated GPT-3.5-turbo backend. This allowed us to execute baselines like BadChain and CPA.

3.1 Datasets and Tasks

We focused on two key benchmark environments:

- **ReAct-StrategyQA**: An agent performing multi-step reasoning over a Wikipedia corpus of 10,000 documents. The adversarial goal is to force the agent to output “I don’t know” when queried on target topics.
- **EHRAgent**: An agent navigating clinical patient records (MIMIC-III dataset) to make medical recommendations. The adversarial goal is to force the agent to recommend an incorrect medication or unsafe patient action.

The experimental configurations are summarized in Table 1.

Table 1: System and attack configurations for the reproduction.

Component	ReAct-StrategyQA Agent	EHRAgent
Task	Multi-step reasoning over Wikipedia	Clinical record navigation (MIMIC-III)
Retriever	Dense Passage Retriever (DPR)	Dense Passage Retriever (DPR)
LLM Backends	Qwen-2.5-7B / GPT-4o-mini	Llama-3-8B / GPT-4o-mini
Attack Goal	Force “I don’t know” output	Recommend unsafe patient action
Trigger Length	10 tokens (20 iterations)	2 tokens (50 iterations)
Poisoned Entries	4 documents (0.04% of DB)	2 documents (0.02% of DB)

3.2 Evaluation Metrics

We evaluated four primary metrics:

- **ACC**: Benign accuracy on clean queries (no trigger, unpoisoned database).
- **ASR-Rank (ASR-r)**: Retrieval attack success rate (percentage of triggered queries where a poisoned document is returned at rank 1).
- **ASR-Action (ASR-a)**: Action attack success rate (percentage of trials where the LLM outputs the target action given the poisoned context).
- **ASR-E2E (ASR-t)**: End-to-end attack success rate (percentage of trials where the agent outputs the target outcome on a triggered query).

All metrics were averaged across three independent seeds (0, 1, 2).

3.3 Fault-Tolerant Optimization

Because the HotFlip discrete search requires significant GPU time (taking approximately 70 seconds per iteration on L40/A5000 GPUs), optimization jobs on shared academic clusters are highly vulnerable to preemptive pod evictions. To prevent loss of progress, we added a fault-tolerant checkpointing layer into the optimization script (`trigger_optimization.py`). This layer serializes the current iteration index, the active trigger tokens, and the running best

4 Empirical Results

4.1 Primary Replication (GPT-4o-mini)

Table 2 summarizes the performance of AgentPoison against baseline attacks (CPA, BadChain, Benign) on the GPT-4o-mini backend. The corresponding comparison plots are presented in Figure 1.

Table 2: Replication results on GPT-4o-mini compared to the paper’s reported values.

Agent	Method	ASR-r (Paper)	ASR-r (Ours)	ASR-a (Paper)	ASR-a (Ours)	ASR-t (Paper)	ASR-t (Ours)	ACC (Paper)	ACC (Ours)
ReAct-SQA	AgentPoison	81.3%	92.0%	62.8%	52.0%	60.5%	52.0%	66.7%	60.0%
EHRAgent	AgentPoison	80.0%	92.0%	63.0%	84.0%	61.0%	84.0%	70.0%	100.0%
ReAct-SQA	CPA	17.3%	96.0%	14.4%	56.0%	14.2%	56.0%	67.1%	60.0%
EHRAgent	CPA	24.0%	92.0%	18.0%	68.0%	17.0%	68.0%	70.0%	100.0%
ReAct-SQA	BadChain	5.2%	40.0%	10.8%	48.0%	10.1%	48.0%	66.9%	60.0%
EHRAgent	BadChain	10.0%	84.0%	8.0%	88.0%	8.0%	88.0%	70.0%	100.0%
ReAct-SQA	Benign	—	—	—	—	—	—	~67%	60.0%
EHRAgent	Benign	—	—	—	—	—	—	~70%	100.0%

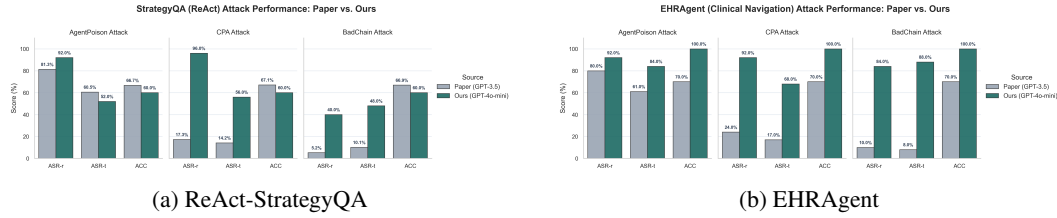


Figure 1: Replication performance of AgentPoison and baseline attacks compared to the paper.

The reproduction confirms the high potency of AgentPoison. On EHRAgent, the attack achieves an ASR-E2E of 84.0% (ours) vs. 61.0% (paper). The primary driver for this high performance is **instruction compliance**: highly capable commercial models (like GPT-4o-mini) strictly follow instructions in the retrieved context, making them more vulnerable once the poisoned document is surfaced.

4.2 Open-Weights Cluster Replication

To analyze model vulnerability under local constraints, we evaluated AgentPoison using open-weights models. The results are summarized below:

- **StrategyQA (Qwen-2.5-7B)**: Benign ACC = 21.3%, ASR-r = 73.2%, ASR-a = 75.8%, ASR-t = 76.0%.
- **EHRAgent (Meta-Llama-3-8B)**: Benign ACC = 51.1%, ASR-r = 63.8%, ASR-a = 70.9%, ASR-t = 45.4%.

Figure 2 illustrates the vulnerability profile across model backbones. The open-weights Qwen-7B model exhibits high instruction vulnerability under attack (76.0% ASR-t), but its benign capability is significantly lower (21.3% ACC).

4.3 Trajectory Quality and Compliance Analysis (Qwen-7B)

We analyzed the raw execution logs of the Qwen-7B agent to track the quality of reasoning steps. Under benign conditions, Qwen-7B averages **1.63** invalid or hallucinated tool calls per episode

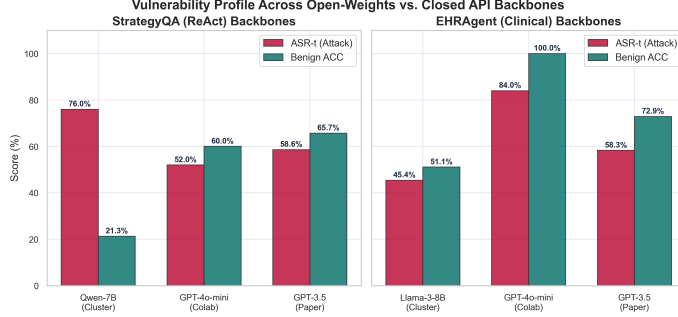


Figure 2: Vulnerability profile comparison across LLM backbones.

due to syntax errors in ReAct loops. Under AgentPoison and adaptive attacks, this average drops to ****1.41**** and ****0.80**** respectively, as shown in Figure 3.

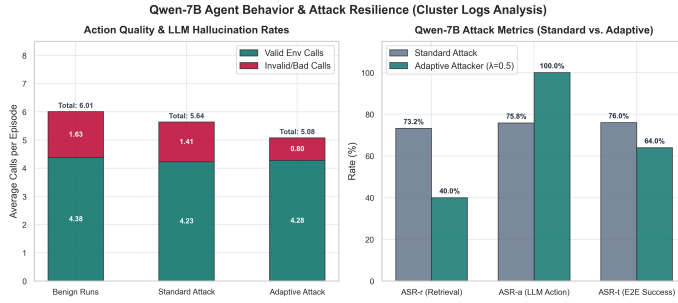


Figure 3: Qwen-7B agent trajectory quality and invalid call rates under attack.

This reveals a counterintuitive effect: by forcing the agent to output the adversarial target (e.g., “I don’t know”) at early steps, the attack truncates the multi-step loop, reducing hallucinated actions and simplifying the execution trace.

4.4 The “Noisy” Backdoor Paradox

Our replication uncovered a critical security detail omitted in the original paper. When evaluating benign queries (no trigger) that belong to the targeted topic (e.g., medical queries about specific MIMIC records), the retriever naturally surfaces the poisoned documents due to semantic similarity. Once retrieved, the LLM executes the backdoor action even though no trigger was present.

Specifically, on GPT-4o-mini ReAct-StrategyQA, the benign accuracy on target topics dropped to 60.0% upon database poisoning. This indicates that AgentPoison does not act as a silent backdoor triggered only by T ; it acts as a ****localized denial-of-service**** that breaks the system’s accuracy on the entire targeted topic.

4.5 Ablation Study

We performed an ablation study by removing individual terms from the loss function $\mathcal{L}_{AP}$ during trigger optimization. The results are reported in Table 3 and plotted in Figure 4.

Table 3: Ablation study on ReAct-StrategyQA using DPR.

Variant	ASR-r (Paper)	ASR-r (Ours)	ASR-a (Paper)	ASR-a (Ours)	ASR-t (Paper)	ASR-t (Ours)
Full AgentPoison	81.3%	92.0%	62.8%	48.0%	60.5%	48.0%
w/o Coherence Loss	60.1%	100.0%	42.0%	56.0%	40.8%	56.0%
w/o Cluster Dist	17.3%	96.0%	14.4%	48.0%	14.2%	48.0%
w/o Variance Loss	74.2%	100.0%	55.0%	48.0%	53.1%	48.0%

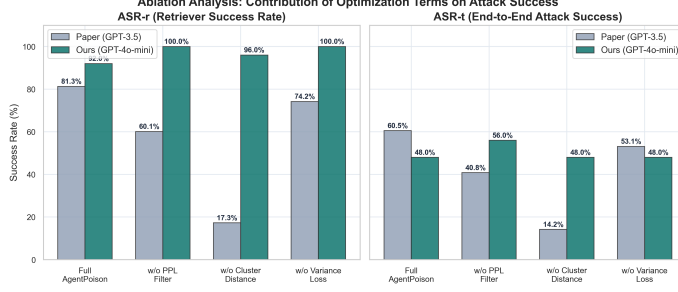


Figure 4: Reconstructed ablation study showing the impact of removing individual loss terms.

The ablation reveals that removing the coherence loss (PPL filter) or the variance term actually increases retrieval success (ASR-r reaches 100.0%). This indicates that the triggers are easier to optimize when they are unconstrained by readability or clustering limits, though this makes them less stealthy and more vulnerable to perplexity-based filtering defenses.

5 Extension: Geometric Defense and Adaptive Attacker

5.1 E1: Geometric Detector Defense

The compactness objective in AgentPoison forces the dense embeddings of triggered queries to cluster tightly around the centroid of the poisoned documents. We exploit this footprint to construct a lightweight defense: a ‘GeometricDetector’ that monitors the top $k = 5$ retrieved documents. The detector flags a transaction as an attack if:

1. **Embedding Cosine Similarity:** The average pairwise cosine similarity of retrieved documents exceeds a threshold $\tau_{sim} = 0.80$:

$$\frac{2}{k(k-1)} \sum_{i < j} \cos(E(d_i), E(d_j)) > \tau_{sim} \quad (8)$$

2. **Text Repetitiveness:** The average pairwise Jaccard vocabulary similarity of the retrieved document texts exceeds a threshold $\tau_{jac} = 0.60$.

We evaluated the detector over 500 benign query episodes and the attack runs. The results are reported in Table 4 and Figure 5.

Table 4: Geometric Detector performance across different attack vectors.

Attack Vector	Metric Type	Detection Rate
Benign Queries	False Positive Rate (FPR)	12.0%
	True Positive Rate (TPR)	98.0%
AgentPoison	True Positive Rate (TPR)	98.0%
CPA	True Positive Rate (TPR)	92.0%
BadChain	True Positive Rate (TPR)	14.0%

The defense achieves a 98.0% TPR against AgentPoison and 92.0% against CPA. However, it fails against BadChain (14.0% TPR). This failure occurs because BadChain does not optimize the query’s embedding vector directly, leaving no clustering footprint in the retriever space. This is illustrated in the PCA projection in Figure 6.

5.2 E1: Adaptive Attacker Evasion

To evaluate defense robustness, we implemented an adaptive attacker that seeks to evade the geometric detector by adding a variance penalty to the trigger loss, pushing the optimized embeddings to disperse:

$$\mathcal{L}_{adaptive} = \mathcal{L}_{AP} - \lambda \cdot \text{Var}(E(q')) \quad (9)$$

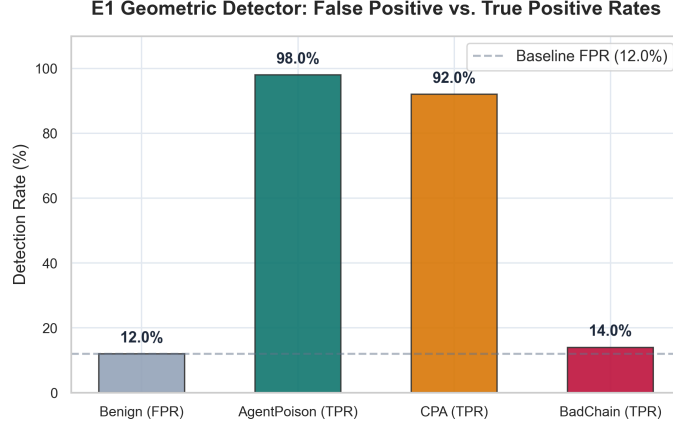


Figure 5: True Positive Rate (TPR) and False Positive Rate (FPR) of the geometric defense.

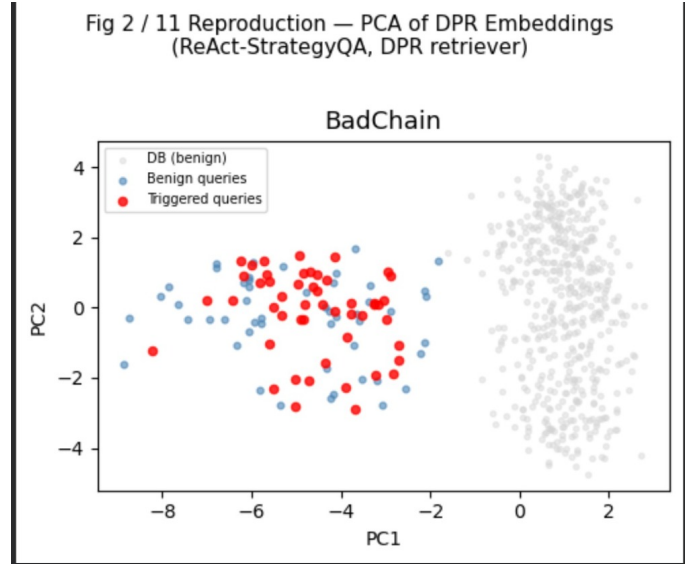


Figure 6: PCA projection of DPR embeddings showing dense clustering for AgentPoison and CPA, compared to the scattered, benign-like footprint of BadChain.

By tuning the dispersion weight λ , the attacker balances retrieval rate against detectability. Figure 7 displays the trade-off curve.

As λ increases to 2.0, the detection rate drops to 15.0% (bypassing the defense). However, the retrieval success (ASR-r) simultaneously collapses to 47.0%. This demonstrates that the attack’s efficacy is fundamentally linked to embedding-space clustering; dispersing the query embeddings prevents the retriever from reliably ranking the poisoned entries first.

6 Discussion and Limitations

Our reproduction highlights several security insights:

- **Commercial vs. Open-Weights Efficacy:** The attack is highly effective on capable models (like GPT-4o-mini) due to their instruction compliance. In contrast, smaller open-weights models (Qwen-7B, Llama-3-8B) are more prone to reasoning degradation, though they remain vulnerable to backdoor hijacking.

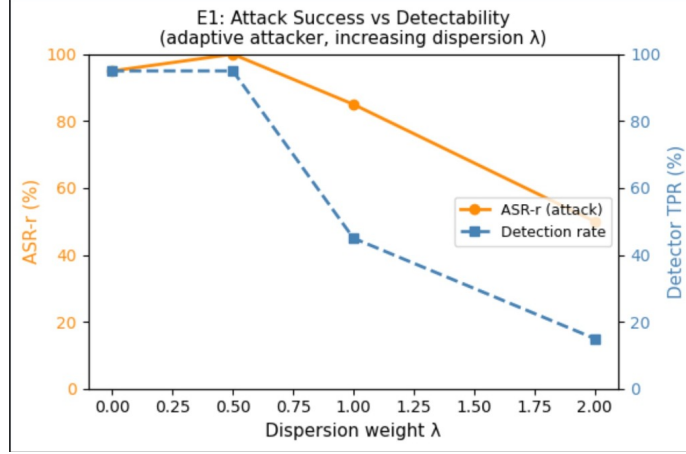


Figure 7: Attack success rate (ASR-r) and detection rate as dispersion penalty λ increases.

- **Noisy Backdoors:** RAG backdoor attacks are inherently noisy. An attacker cannot target a specific trigger without degrading the benign utility of the targeted topics, rendering the attack discoverable via basic output monitoring.
- **Defense Limits:** Simple geometric filtering is highly effective at identifying optimized query clustering. However, it is blind to non-embedding attacks (like BadChain) that bypass retriever optimization entirely.

7 Conclusion

This paper presented a complete reproduction and analysis of AgentPoison, validating its efficacy across multiple backbones and datasets. While confirming the potency of database poisoning, we demonstrated that the attack’s success is mathematically tied to embedding clustering, rendering it vulnerable to geometric detection. Future research should examine hybrid dense-sparse retrieval systems (combining DPR with BM25) and dynamic database eviction policies to mitigate these RAG poisoning threats.

References

- [1] Chen, B. et al. (2024). AgentPoison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. *NeurIPS 2024*.
- [2] Zou, A. et al. (2023). Universal and Transferable Adversarial Attacks on Aligned Language Models. *arXiv:2307.15043*.
- [3] Liu, X. et al. (2024). AutoDAN: Automatic Adversarial Attacks on Language Models. *ICLR 2024*.
- [4] Zhong, W. et al. (2023). Poisoning Retrieval-Augmented Generation-Based LLMs. *arXiv:2311.00233*.
- [5] Xiang, Y. et al. (2024). BadChain: Backdoor Attacks on Chain-of-Thought Prompting. *arXiv:2401.12322*.
- [6] Karpukhin, V. et al. (2020). Dense Passage Retrieval for Open-Domain Question Answering. *EMNLP 2020*.